

Semana 2: Sintaxis de Python básica — variables, tipos de datos y entrada/salida

Nivel: Básico · Prerrequisitos: Semana 1 · Versión: v1.0 · Fecha: 2026-08-31

Objetivos de aprendizaje

1. Escribir y ejecutar sintaxis Python válida para crear variables, respetando las reglas de nombrado del lenguaje.
2. Identificar los cuatro tipos de datos primitivos de Python (`int`, `float`, `str`, `bool`) y predecir el tipo resultante de una operación dada.
3. Usar `input()` para leer datos del teclado y explicar por qué el resultado siempre llega como texto (`str`).
4. Convertir explícitamente entre tipos de datos (`int()`, `float()`, `str()`) para evitar errores de tipo, y explicar cuándo esa conversión es necesaria.
5. Construir un programa completo de entrada → procesamiento → salida usando `input()`, operadores y `print()` con f-strings.

1. De la caja etiquetada a la sintaxis real

En la Semana 1 vimos la variable como un **casillero etiquetado**: un nombre fijo (`EDAD`) que puede contener distintos valores en el tiempo (`30`, luego `31`). Hoy esa idea deja de ser una analogía y se convierte en sintaxis ejecutable.

En Python, crear una variable es una sola línea:

```
edad = 30
```

A la izquierda, la etiqueta del casillero (`edad`). A la derecha, lo que se guarda dentro (`30`). El signo `=` no es "igual a" como en matemáticas — es **asignación**: "toma el valor de la derecha y guárdalo en el nombre de la izquierda". Esta es exactamente la misma distinción entre `=` y `==` que ya adelantamos en la Semana 1 con el ejemplo del paraguas.

Reglas de nombrado (lo que Python exige, no es opcional):

- Solo letras, números y guión bajo (`_`); no puede empezar con número.
- Distingue mayúsculas de minúsculas: `Edad` y `edad` son dos casilleros distintos.

- No se pueden usar palabras reservadas del lenguaje (`if`, `for`, `while`, `True`, etc.).

Convención (lo que la comunidad de Python recomienda, sí es buena práctica): nombres en `snake_case` y descriptivos — `precio_total` en vez de `pt` o `x`. Un nombre claro es documentación gratis; en ingeniería de software esto se llama código **autoexplicativo**, y ahorra tiempo a cualquiera que lea el código después (incluido tú mismo, en tres semanas).

2. Los cuatro tipos de datos primitivos

Volvamos a la analogía del casillero: no todos los casilleros son iguales. Un casillero para guardar líquidos (una botella) no sirve para guardar harina suelta (necesita un frasco). El **tipo de dato** es exactamente eso — la forma del contenido, que determina qué operaciones tienen sentido sobre él.

TIPO	NOMBRE EN PYTHON	EJEMPLO	ANALOGÍA
Número entero	<code>int</code>	<code>30</code> , <code>-4</code> , <code>0</code>	Contar personas: no existen "2.5 personas".
Número decimal	<code>float</code>	<code>1.75</code> , <code>-0.5</code>	Medir estatura o peso: sí admite fracciones.
Texto	<code>str</code>	<code>"Ana"</code> , <code>"30"</code>	Un letrero: aunque diga "30", son letras, no un número.
Verdadero/falso	<code>bool</code>	<code>True</code> , <code>False</code>	Un interruptor: solo dos posiciones posibles.

Función clave para explorar esto: `type()`, que le pregunta a Python "¿qué tipo de casillero es este?".

```
edad = 30
estatura = 1.75
nombre = "Ana"
es_estudiante = True

print(type(edad))           # <class 'int'>
print(type(estatura))       # <class 'float'>
print(type(nombre))         # <class 'str'>
print(type(es_estudiante))  # <class 'bool'>
```

El truco que confunde a todo principiante: `"30"` (con comillas) y `30` (sin comillas) se ven casi iguales, pero son tipos distintos. `"30"` es una etiqueta de texto que **dice** treinta; `30` es la cantidad treinta, y solo con la segunda puedes hacer `30 + 1`. Esto no

es un detalle menor: es la causa número uno de errores en el próximo bloque (entrada de datos).

3. Operadores: lo que se puede hacer con cada tipo

Aritméticos (sobre `int` / `float`): `+` `-` `*` `/` `//` `%` `**`

```
print(7 / 2)    # 3.5    → división siempre da float
print(7 // 2)   # 3      → división entera (descarta el resto)
print(7 % 2)    # 1      → módulo: el resto de la división
print(2 ** 3)   # 8      → potencia
```

Comparación (dan como resultado un `bool`): `==` `!=` `>` `<` `>=` `<=` — ya los usamos en la Semana 1 dentro de un `if`.

```
print(18 >= 18)    # True
print("Ana" == "ana") # False → Python distingue mayúsculas también en texto
```

Un detalle que conecta con el bloque de "decisión" de la Semana 1: un `bool` no es una curiosidad aislada, es literalmente el tipo de dato que vive dentro de cada `if`. `edad >= 18` no es solo una condición abstracta: es una expresión que Python evalúa y convierte en un valor concreto de tipo `bool`, que luego el `if` usa para decidir el camino.

4. Entrada y salida: `input()` y `print()`

Hasta ahora todos los datos de nuestros programas venían "cableados" en el código (`edades = [12, 17, ...]`). Un programa real necesita recibir datos de quien lo usa.

```
nombre = input("¿Cómo te llamas? ")
print(f"Hola, {nombre}")
```

`input()` muestra el mensaje entre paréntesis, pausa el programa, y **lo que la persona escriba llega siempre como `str`** — sin excepción, aunque escriba solo números. Es como un mesero que, sin importar lo que pidas, siempre lo anota en un papel: el papel (texto) es el medio, no el contenido real.

```
edad = input("¿Cuántos años tienes? ")
print(type(edad)) # <class 'str'> — ¡aunque la persona haya tecleado "18"!
```

Esto significa que `edad + 1` con este `edad` **falla**, porque Python no sabe sumar texto con número (`TypeError`). La solución es la **conversión de tipos**:

```
edad = int(input("¿Cuántos años tienes? ")) # convierte el texto a int de inmediato
print(edad + 1) # ahora sí funciona
```

Las tres conversiones más comunes: `int("30")` → `30`, `float("1.75")` → `1.75`, `str(30)` → `"30"`. Todas fallan con un error claro (`ValueError`) si el texto no representa realmente ese tipo — por ejemplo, `int("hola")` truena, porque "hola" no es un número.

print() con f-strings (ya las usamos en la Semana 1, ahora las formalizamos): anteponer `f` a un string permite insertar variables directamente entre llaves `{}`, sin concatenar manualmente con `+`.

```
nombre = "Ana"
edad = 30
print(f"{nombre} tiene {edad} años") # f-string: legible
print(nombre + " tiene " + str(edad) + " años") # equivalente, pero más frágil (hay que convertir a mano)
```

5. Por qué esto importa para IA/ML

Cada modelo de Machine Learning, sin excepción, procesa datos que primero pasaron por exactamente esta misma disciplina de tipos. Una imagen no es "una foto" para el modelo: es una matriz de números (`float`). Una reseña de producto no es "texto libre" para el modelo: eventualmente se convierte en números también. El primer error que comete casi cualquier persona empezando en ciencia de datos es intentar hacer una operación matemática sobre una columna que, sin que lo note, Python sigue tratando como texto (`str`) — el mismo `TypeError` de `edad + 1` que acabamos de ver, a mayor escala. Cuando en semanas futuras uses `pandas` o `NumPy`, la primera pregunta de diagnóstico ante cualquier error extraño va a ser la misma que hoy: **"¿de qué tipo es realmente este dato?"**.

6. Ejemplo práctico: calculadora de IMC

Problema: pedir peso (kg) y estatura (m) por teclado, calcular el Índice de Masa Corporal ($\text{IMC} = \text{peso} / \text{estatura}^2$) y mostrar una clasificación.

Pseudocódigo:

INICIO

```
peso = preguntar el peso en kilogramos
estatura = preguntar la estatura en metros
imc = peso / (estatura * estatura)
SI imc es menor a 18.5 ENTONCES
    mostrar "Bajo peso"
SI NO SI imc es menor a 25 ENTONCES
    mostrar "Peso normal"
SI NO
    mostrar "Sobrepeso"
mostrar el valor de imc
```

FIN

Del pseudocódigo al código real (Python)

```
# ENTRADA: input() siempre da texto, así que convertimos de inmediato con float()
# (float, no int, porque el peso y la estatura admiten decimales: 68.5 kg, 1.72 m).
peso = float(input("Ingresa tu peso en kilogramos: "))
estatura = float(input("Ingresa tu estatura en metros: "))

# PROCESAMIENTO: operador ** para la potencia, igual que en la sección 3.
imc = peso / (estatura ** 2)

# DECISIÓN: "SI NO SI" del pseudocódigo se traduce a elif en Python.
# elif evita anidar un if dentro de otro if – se lee como una sola cadena de casos.
if imc < 18.5:
    categoria = "Bajo peso"
elif imc < 25:
    categoria = "Peso normal"
else:
    categoria = "Sobrepeso"

# SALIDA: f-string con formato :.1f para redondear a 1 decimal (más legible que
23.437...).
print(f"Tu IMC es {imc:.1f} ({categoria})")
```

Con `peso = 68`, `estatura = 1.72`: `imc = 68 / (1.72 ** 2) = 22.99...` → `Tu IMC es 23.0 (Peso normal)`.

Punto de confusión clásico: `elif` no es lo mismo que escribir un `if` nuevo justo después de otro. Con `if` / `elif` / `else` encadenados, en cuanto una condición se cumple, Python evalúa esa rama y **se salta el resto de la cadena** — no sigue

comprobando las siguientes. Si en cambio se escriben varios `if` sueltos, Python evalúa todos y cada uno, sin importar si uno anterior ya se cumplió, lo que puede producir resultados que se pisan entre sí.

Segundo ejemplo: promedio de tres calificaciones

```
nota1 = float(input("Nota 1: "))
nota2 = float(input("Nota 2: "))
nota3 = float(input("Nota 3: "))

promedio = (nota1 + nota2 + nota3) / 3    # SECUENCIA

if promedio >= 6.0:                        # DECISIÓN (recordando la Semana 1)
    resultado = "Aprobado"
else:
    resultado = "Reprobado"

print(f"Promedio: {promedio:.2f} – {resultado}")
```

7. Notas de vigencia técnica (2026)

La sintaxis de variables, tipos primitivos, `input()` / `print()` y f-strings es estable desde hace años y sigue siendo la forma estándar y recomendada de escribir Python en 2026 — no hay ninguna parte de este contenido en camino a quedar obsoleta. Se mantiene la recomendación de la Semana 1: **Python 3.12+** en un entorno en línea (Jupyter, Google Colab o Replit) para evitar fricción de instalación. Un detalle de estilo moderno: se prioriza el uso de f-strings (`f"{variable}"`) sobre el método antiguo `.format()` o la concatenación con `+`, porque son más legibles y son la convención dominante en el código Python actual.

8. Errores comunes de principiante

- Olvidar que `input()` siempre devuelve `str`, e intentar sumar directamente (`edad + 1`) truena si `edad` viene de `input()` sin convertir).
- Confundir `"30"` (texto) con `30` (número) porque se ven parecidos al imprimirlos.
- Usar `=` cuando se quería comparar (`if x = 5` es un error de sintaxis en Python; lo correcto es `if x == 5`).
- Encadenar varios `if` sueltos cuando la lógica pedía `elif`, provocando que se evalúen ramas que ya no correspondían.

- Nombrar variables con una sola letra (`x`, `p`) en programas que ya tienen algo de lógica, dificultando releerlos después.

9. Ejercicio propuesto

Escribe un programa en Python que funcione como una pequeña caja registradora:

```
Pide el precio de un producto (con decimales) y la cantidad comprada (un número entero).
Calcula el total (precio × cantidad).
Si el total supera los 50, aplica un 10% de descuento sobre el total.
Muestra el total final con 2 decimales, indicando si se aplicó descuento o no.
```

Pista: vas a necesitar `float()` e `int()` para las dos entradas, un `if/else`, y una f-string con formato `:.2f` para el resultado, igual que en el ejemplo del IMC.

10. Autoevaluación

1. ¿Por qué `edad = input("¿Cuántos años tienes? ")` no permite hacer `edad + 1` directamente? — Porque `input()` siempre devuelve un dato de tipo `str` (texto), sin importar lo que la persona haya tecleado; Python no puede sumar texto con número sin conversión explícita.
2. ¿Cuál es la diferencia entre `"30"` y `30`? — `"30"` es un dato de tipo `str` (texto, entre comillas); `30` es un dato de tipo `int` (número). Se ven parecidos al imprimirse pero admiten operaciones distintas.
3. ¿Qué hace exactamente el operador `=` en `total = precio * cantidad`? — Asigna: calcula el valor de la derecha y lo guarda en el nombre de la izquierda. No es una comparación de igualdad (para eso existe `==`).
4. Da el tipo de dato resultante de cada expresión: a) `7 / 2` b) `7 // 2` c) `"7" + "2"` d) `7 == 2`. — a) `float` (3.5), b) `int` (3), c) `str` ("72", concatenación de texto, no suma), d) `bool` (False).
5. ¿Por qué se usa `elif` en vez de una cadena de varios `if` sueltos en la calculadora de IMC? — Porque con `elif` en cuanto se cumple una condición se ejecuta esa rama y se salta el resto de la cadena; con varios `if` sueltos, Python evaluaría todos, lo que podría hacer que más de una rama se ejecute o que el resultado no sea el esperado.
6. V/F: "En Python, `edad` y `Edad` son la misma variable." — Falso: Python distingue mayúsculas de minúsculas, son dos casilleros distintos.

7. ¿Qué conversión de tipo usarías para leer un peso con decimales desde `input()`, y por qué no `int()`? — `float()`, porque el peso puede tener decimales (68.5 kg); `int()` los descartaría o directamente fallaría si el texto trae un punto decimal en algunos casos de uso.
8. Explica con tus propias palabras por qué esta disciplina de tipos importa para Machine Learning, sin usar jerga de programación. — Porque los modelos de IA finalmente procesan todo como números; si un dato que debería ser numérico sigue "disfrazado" de texto, la operación falla exactamente igual que `edad + 1` sin convertir, solo que a mayor escala y con datos reales.
9. En el ejemplo de la caja registradora (ejercicio propuesto), ¿qué tipo de dato debería tener la variable "cantidad comprada" y por qué? — `int`, porque se compran unidades completas de producto (no tiene sentido comprar "2.5 productos" en este contexto), a diferencia del precio que sí admite decimales.

11. Verificación de dependencias

Este contenido depende únicamente de la Semana 1 (pensamiento computacional): reutiliza directamente la idea de variable como casillero etiquetado, la distinción entre `=` y `==`, y los bloques de secuencia/decisión ya vistos con pseudocódigo — no introduce ningún concepto matemático ni de programación que no haya sido cubierto o preparado explícitamente antes. Es prerequisite directo de la Semana 3 (estructuras de control y funciones), que asume que ya se sabe declarar variables, distinguir tipos de datos y leer/escribir datos por consola.

Aprende con Gutty IA — curso semanal de programación, IA y Machine Learning. Material de portafolio público.